One alert.
Three architecture decisions.

A 10-minute walkthrough of Threshold Monitor. Show the behavior, point to the code, explain the tradeoff.

> “I’ll follow one alert from detection to an operator’s acknowledgement, then show three decisions that make that behavior understandable and testable.”

Time	Screen	Your purpose
0:00–2:00	System overview	Assign responsibilities: sensor detects, relay persists, monitor displays, control room responds.
2:00–5:00	Live simulator	Show one crossing, delivery, and server-confirmed acknowledgement.
5:00–6:00	Code: crossing.ts	Pure detection separates domain decisions from clocks and sockets.
6:00–7:30	Code: Redux state → Actions & reducers	Make state ownership explicit; show one acknowledgement update.
7:30–9:00	Code: outbox.ts → routing.ts	Distinguish socket writes from acceptance; connect retries to UUID deduplication.
9:00–10:00	System overview	Name current limits and the next concrete reliability improvement.

Before people arrive

- Open the offline guide and keep this sheet on a second screen or printed.
- Use English or German consistently. The Code Explorer supports both; this speaking script is English.
- In Live Simulator, choose **Cross both directions**, then **Restart**. Saved runs survive a reload.
- Return to Overview. Check screen sharing, readable browser zoom, and that the simulator map and alarm list are visible.
- The guide is an offline educational model. A real relay or login is unnecessary for this walkthrough.

If you only get five minutes

Use one minute for the map, two for the crossing and acknowledgement, one for the outbox code, and one for state ownership and limits. Keep the remaining examples for questions.

Keep the visible guide focused on the system. These notes are separate presenter material.

Tell the story while stepping

0:00 · Overview — point to each component

> “The sensor detects transitions in the measurements. The relay validates and stores alerts, then distributes them. The monitor shows readings and edits settings. The control room lets an operator acknowledge an alarm. SQLite preserves alerts, acknowledgements, and settings across relay restarts.”

Transition: “Let’s follow one alert. This simulator models the message flow; it does not connect to the running relay.”

2:00 · Simulator — choose Cross both directions → Restart

Threshold: **25°C**. Hysteresis width: **0.6°C**. Edges: **24.7°C and 25.3°C**. Counts below are total Step clicks since Restart.

Step count	What you see	What to say
1–3	Sample 1: 24°C reaches the monitor.	“The first reading establishes state. Starting on one side does not itself create an alert.”
4–9	25.1°C, then exactly 25.3°C. No alert.	“Crossing the central threshold is insufficient. Even touching the upper edge creates no alert: the comparison is strict.”
10 · pause	25.4°C creates an upward alert. Sensor stage.	“Now the reading exceeds the upper edge. The sensor creates an alert and keeps it pending in the outbox.”
11 · pause	Relay stores the alert and returns accepted.	“Storage and acceptance are separate from the sensor’s socket write. The sensor can now remove this UUID from its outbox.”
12 · pause	The alert appears in both browser apps.	“Each app updates its own view of the same server record.”

Acknowledge, then three more Step clicks

Click **Acknowledge**: the request is sent, but the alarm is not yet confirmed.

Step 1: relay stores the first acknowledgement. **Step 2:** relay broadcasts confirmation. **Step 3:** browser copies show “Acknowledged by Alex”. Routes stay highlighted between clicks.

> “The click requests a change. The UI shows success after server confirmation. The operator’s identity comes from the authenticated session, rather than a name supplied in the request.”

Stop the main demo here. The downward crossing is optional. Transition: “Now I’ll show three code decisions behind this behavior.”

Show a few lines. Explain the choice.

5:00 · Detection & delivery → crossing.ts

> “Detection is a small state machine. Previous state, a reading, and settings determine the result. There are no sockets or clocks inside it, so I can test exact boundaries and noisy sequences directly.”

Point to: the upper and lower comparisons, and the returned next state. Connect them to 25.3°C versus 25.4°C in the demo.

Tradeoff: hysteresis deliberately delays the opposite transition until the other edge is crossed. It suppresses noise at the cost of a wider switching band.

Do not read every branch. One boundary example establishes the design.

6:00 · Shared state → Redux state → Actions & reducers

> “The monitor uses Redux Toolkit for confirmed settings, sensor information, and normalized alerts. The state shape makes ownership visible. An acknowledgement action updates the existing record by ID; selectors derive the view.”

Point to: `settings.current`, `alerts.ids`, `alerts.entities`, then the acknowledgement guard and assignment in the alert reducer.

Explain the boundary: “Form drafts stay local. Transient readings use a bounded external store. The control room has a smaller alarm model and uses `useReducer`.”

Tradeoff: there are multiple state mechanisms to understand. Each has a clear owner and lifetime. Redux could handle readings; the separate store is an intentional design choice, not a Redux limitation.

7:30 · Detection & delivery → outbox.ts → routing.ts

> “A successful socket write doesn’t prove the relay stored the alert. Sending moves it into pending. Acceptance removes it; a deadline returns it to the retry queue. Retries keep the same UUID, so the relay can acknowledge an existing record without broadcasting a duplicate.”

Point to: pending deadlines, `accept`, `expire`, then the existing-record branch in relay routing.

Tradeoff: the outbox is bounded and in memory. Process loss or queue overflow can lose alerts. This is conditional at-least-once delivery, not an unconditional delivery guarantee.

9:00 · Return to Overview — close

> “The current design targets one sensor and one relay. My next reliability improvement would be a durable sensor outbox. Browser recovery also needs replay after a cursor or an acknowledgement lookup, because the welcome snapshot is bounded. The key decisions are explicit state ownership, pure domain logic, and confirmation at the right boundary.”

Pause for questions. Keep the draft form, hooks, and database examples as backup.

Short answers, with a code example ready

Why Redux here, but not everywhere?

The monitor has shared settings, normalized alerts, and derived views. Redux Toolkit gives those updates explicit actions and selectors. The control room's smaller alarm list uses a reducer. Local editing state stays with its component.

Why a separate readings store?

Measurements are transient and bounded. The chart and readout subscribe through `useSyncExternalStore`. Redux could support the update rate; the reason is ownership and lifetime, not that every Redux subscriber must re-render.

Show a React UX decision

Open **Draft vs. confirmed state**: "Typing 25 must not briefly send 2 as a shared setting. The component holds a string draft, validates it on commit, and requests the update. Redux receives the server-confirmed numeric value." The round trip introduces a delay; the active value remains visible.

What if the relay stored an alert but acceptance was lost?

The sensor retries the same UUID. The relay recognizes the stored record and accepts it without sending another new-alert broadcast. Open **routing.ts**.

What if two operators acknowledge the same alarm?

The server takes identity from the session. A conditional database update preserves the first acknowledgement. Open **alertStore.ts**; the client's click alone is not authoritative.

Does reconnect recover everything?

No. Welcome includes at most 100 recent alerts, not a complete replay. An older retained control-room alarm can miss its acknowledgement. Readings are transient, and the sensor outbox is not durable.

Why SQLite? How would this scale?

A single relay and local database keep this deployment simple. Several relays would need a shared store, distribution between relay instances, and atomic deduplication and first-acknowledgement behavior.

Rehearse twice

1. **First pass:** use a timer. Reach the first alarm in 12 Step clicks. Acknowledge, then use three more Steps. Switch to the three code examples without searching.
2. **Second pass:** speak from the screen, using these notes only for timing. If you pass five minutes before opening code, shorten the demo explanation.
3. **Before presenting:** restart the simulator, return to Overview, and keep the offline guide and this PDF available. If screen sharing fails, explain sensor → relay → browsers verbally using the same story.

For more questions: interview-questions.pdf. Reference: architecture.md. These notes describe the current implementation; proposed reliability improvements are not implemented.

How would you extend this system?

These answers describe proposed work. The features below are not implemented.

Answer pattern: clarify the behavior → identify the owner → explain the data flow → name a tradeoff → describe a test. Avoid an estimate until the scope is clear.

1. How would you support multiple sensors?

> "I'd make sensor identity explicit throughout the live data path. Each sensor keeps its own detector and outbox; the relay manages connections, readings, and settings by sensor ID. The monitor can then select a sensor without mixing streams."

Where: alerts already contain `sourceId` and the settings table is keyed by sensor ID. However, reading messages carry no sensor ID, and welcome and sensor status describe a single sensor. Extend `packages/protocol/src/messages.ts`, replace the relay's single-sensor lookup with a registry, and key browser stores and subscriptions by sensor.

Tradeoff / verify: this is a protocol and state-model change, not only a dropdown. Authorize sensor identities and browser access. Test two simultaneous streams, independent settings, a disconnect of only one sensor, and reconnect isolation.

2. How would you add humidity or a real hardware sensor?

> "For replacing the temperature generator, I'd implement the existing DataSource interface and inject it into the agent. The detector already works with numeric values. For humidity, I'd also update metric labels, units, defaults, and shared validation limits."

Where: `sensor/src/data/DataSource.ts`, `sensor/src/agent.ts`, the sensor and monitor configs, and protocol settings validation. Supporting temperature and humidity simultaneously needs metric identity and settings per stream: ordinary readings currently contain only time and value.

Tradeoff / verify: hardware brings missing, invalid, and delayed readings. Decide the adapter's error and timestamp behavior. Test known sample sequences, invalid measurements, disconnect/reconnect, and unit and threshold consistency.

3. What about separate high and low alarm limits?

> "I'd first distinguish an alarm threshold from hysteresis. Today there is one threshold with a noise-suppression band. Independent high and low alarms need explicit alarm rules, rather than relabeling the existing edges."

Where: extend shared settings and validation, migrate stored settings, and update the pure detector, alert metadata, settings form, and chart. Define alarm activation and clearing separately from operator acknowledgement.

Tradeoff / verify: decide what happens on startup, configuration changes, and a jump across both limits. Test equality, noisy edges, repeated samples, and settings changes without synthetic crossings.

Extend the guarantees deliberately

4. How would you add history and filtering?

> “Alert history can query records already in SQLite through an authorized, paginated API. Temperature history needs new storage because ordinary readings are currently transient. I’d keep historical results separate from the live buffer.”

Where: extend the alert store and HTTP handlers; add a monitor history view with loading, error, and empty states. For measurements, add a readings table, bounded batched writes, range queries, and retention or aggregation. Use a stable pagination cursor with a unique tie-breaker.

Tradeoff / verify: storing received measurements cannot recover samples lost before reaching the relay. At 10 Hz, one sensor produces 864,000 rows per day. Test query authorization, time boundaries, concurrent inserts during pagination, and visible gaps. Retention must preserve deduplication information so a late alert retry cannot recreate a deleted alarm.

5. How would you recover every missed acknowledgement?

> “I’d add an ordered, durable change log for alert creation and acknowledgement. On reconnect, clients request changes after the last sequence they applied. A bounded recent-alert snapshot alone cannot guarantee recovery.”

Where: write each mutation and its change-log entry in one SQLite transaction. Extend the protocol and connection layer with bounded replay pages, cursors, and reset responses; reducers apply changes idempotently. Hand replay over to live updates without skipping a sequence.

Tradeoff / verify: retain the cursor with the state it describes; a reload that discards state must also discard that cursor. Expired cursors or a replaced database require an explicit reset. Test an old alarm acknowledged while offline, duplicate replay, live updates during replay, and retention expiry.

6. How would you keep sensor alerts across a crash?

> “I’d persist alerts locally before sending them, then remove or mark them delivered only after relay acceptance. Restart restores the original UUIDs, so relay deduplication still works.”

Where: add a durable sensor outbox adapter and connect it to transport startup, sending, and acceptance. Keep pure queue decisions separate from disk access. Use persistent storage with a documented transaction and flush policy.

Tradeoff / verify: this protects alerts committed locally, not an event lost before its first durable write. Define disk limits, overflow, and full-disk behavior. Test crashes before send and after relay storage but before local deletion; retries must not produce another new-alert broadcast.

More detailed proposals and file mappings: proposed-improvements.md.

Keep new workflows server-authoritative

7. How would you add email, SMS, or webhook notifications?

> “I’d trigger notification work from a committed alert on the relay, not from an open browser. I’d store a notification job in the same database transaction as the alert, then let a worker deliver it with retries.”

Where: extend the persistence boundary with a transactional notification outbox, then add a worker and channel adapter. Use a job identity based on alert, channel, and destination. Keep provider credentials on the server, and define which rules and recipients receive notifications.

Tradeoff / verify: delivery to an external provider is not atomic with a local commit. Prefer provider idempotency where available; otherwise a crash after sending can cause a duplicate. Test provider timeouts, retry backoff, duplicate alerts, and a relay restart between commit and sending.

8. How would you add snooze, mute, or escalation?

> “I’d first separate a local sound preference from a shared operational action. Snooze should not silently acknowledge an alarm. A shared snooze or escalation rule belongs on the server with an owner, expiry time, and audit record.”

Where: a tab-only sound preference can stay local. Shared snooze needs persisted state, authorized commands, protocol updates, and browser reconciliation. Escalation needs a server-owned deadline and restart-safe processing; browser timers are unsuitable when tabs close.

Tradeoff / verify: agree whether new alerts inherit a snooze and what happens when it expires. Test two operator tabs, permission denial, exact expiry, acknowledgement racing with escalation, and a server restart while a deadline is pending.

9. How would you add an admin role or configurable permissions?

> “I’d define capabilities first: viewing measurements, editing settings, acknowledging alarms, and managing users. I’d enforce each action on the server and use the same capabilities to explain available controls in the UI.”

Where: extend user and session authorization, relay handshake and routing checks, and relevant HTTP endpoints. An administrator does not necessarily need a new WebSocket client role; a management HTTP API may be enough. The authenticated session remains the source of operator identity.

Tradeoff / verify: define how existing sessions react when permissions change. Hiding a button does not authorize an action. Test direct forbidden requests, role changes during a session, and audit attribution, as well as the visible UI states.

If asked which feature you would build first

> “I’d prioritize the user’s need. For correctness, reconnect recovery fixes a known stale-acknowledgement gap. For sensor process failures, I’d prioritize the durable outbox. I’d agree acceptance criteria and demonstrate one complete flow before broadening the feature.”